CSE727 Reinforcement Learning

Assignment 1: Lunar Lander

Implement an agent that learns to land safely, analyse the training process, and report experimental findings.

Deadline
23:59, April 18

Weight
20% of final grade

Learning Objectives and Reading

What students should learn and the resources they can consult

Learning objectives

- Understand states, actions, rewards, and episodes
- Implement replay and target networks for stable training
- Manage exploration vs. exploitation with epsilon-greedy policies
- Analyse agent behaviour through learning curves
- Tune hyperparameters and debug failure cases

Additional reading

- Gymnasium main documentation and basic usage tutorials
- LunarLander environment documentation

Assignment Overview

What students are expected to build and why it matters

Problem statement

Build an agent for LunarLander-v3. The goal is to learn a landing policy that brings the spacecraft safely to rest between two flags while using thrust efficiently.

Why this assignment

It combines environment interaction, neural value approximation, exploration, replay, target networks, and experiment design in one practical task.

Solved criterion

The environment is considered solved when the average reward exceeds 200 over 100 consecutive episodes.

Deck roadmap

- Environment details: state space, action space, rewards, and success threshold
- Part A: random baseline and environment setup
- Part B: core agent implementation requirements
- Part C: training, evaluation, and analysis deliverables
- Part D: hyperparameter experiments and written report
- Submission package, grading summary, and suggested readings

Environment Details

LunarLander-v3 setup that students must understand before training

State space (8 dimensions)

- x and y position of the lander
- x and y velocity components
- angle and angular velocity
- left-leg and right-leg contact flags

Action space (4 discrete actions)

- 0 = do nothing
- 1 = fire left orientation engine
- 2 = fire main engine
- 3 = fire right orientation engine

Reward structure

- Small positive/negative reward for moving toward or away from the landing pad
- Crash penalty: −100
- Successful rest: +100
- Each leg contact: +10
- Main engine cost: −0.3 per frame
- Side engine cost: −0.03 per frame

Target: average reward > 200 over 100 episodes

Landing task intuition

The lander starts near the top center with a random initial force. Students should reason about stabilizing angle, reducing descent speed, and aligning with the landing pad before touchdown.

Control challenge

Balancing vertical thrust, lateral correction, and fuel cost makes the reward signal informative but noisy.

Teaching angle

This environment is ideal for introducing exploration vs. exploitation and stability tricks in value-based RL.

Part A — Environment Setup and Baseline

15 points

Tasks

- Install Gymnasium and create the LunarLander-v3 environment
- Implement a random-policy baseline agent
- Run 100 episodes and collect mean reward, episode length, and success rate
- Visualize and save 3–5 episodes as videos or GIFs

Deliverables

- Code for the random agent
- Statistics report with plots
- Sample episode recordings

Grading breakdown

5 pts

Correct environment setup

5 pts

Baseline statistics and summary metrics

5 pts

Visualisations and episode recordings

- https://github.com/NSYSU-ARL/Assignment1
- Follow the setup guide available on eLearn to install Python 3 and the Gymnasium environment on your computer.

Part B — Agent Implementation

40 points

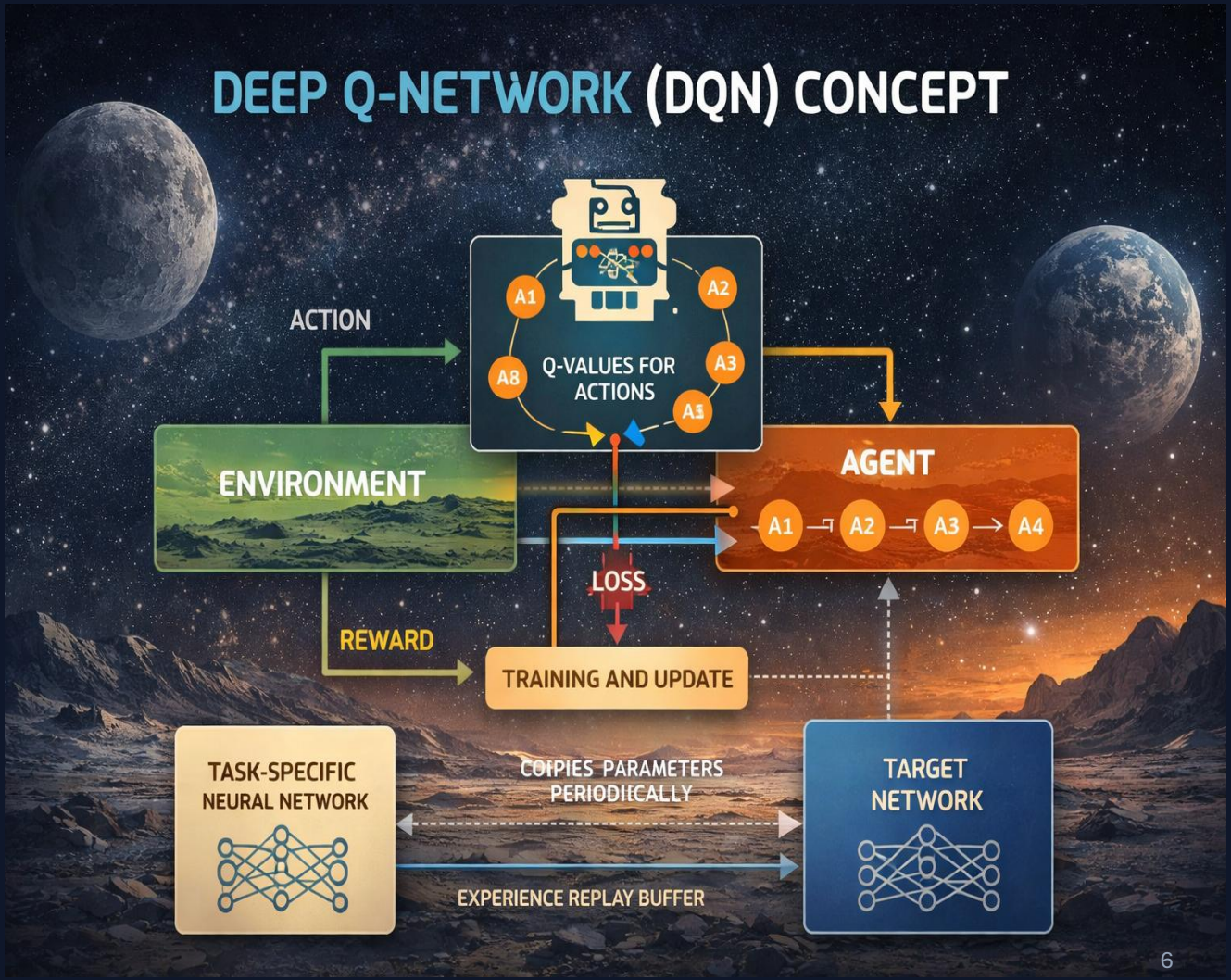
Required components

- All algorithms are acceptable
- Neural network Q-function approximator (if NN-based)
- Experience replay buffer (if NN-based)
- Target network with periodic updates (if NN-based)
- Epsilon-greedy exploration
- Training loop for at least 500 episodes

Deliverables

- Complete Agent implementation
- Training loop with episode management
- Saved model checkpoints

Example:



Part C — Training and Analysis

30 points

Training tasks

- Track episode rewards during training
- Monitor epsilon decay, loss values, and average Q-values
- Plot learning curves with moving averages
- Evaluate the trained policy for 100 no-exploration episodes
- Record 3–5 test episodes of learned behaviour

Expected deliverables

- Plots of rewards, loss, epsilon, and Q-values
- Test performance statistics
- Videos or GIFs of the trained agent

How students are graded

10 pts

Agent shows clear improvement during training

10 pts

Plots and analysis are comprehensive and interpretable

10 pts

Test performance is documented convincingly

Part D — Experimentation and Report

15 points

Hyperparameter experiments

- Students must test at least 3 variations
- Suggested axes: learning rate, batch size, epsilon decay, network depth/width, target update frequency, replay buffer size
- Example values are provided for learning rate and batch size

Report prompts (2–3 pages)

- Effect of epsilon decay on learning speed versus final performance
- Impact of updating the target network too often or too rarely
- Which hyperparameter mattered most
- Observed failure modes during training
- How the learned policy compares with intuition about optimal landing

Instructor-facing value

Beyond implementation

This part checks whether students can reason about the stability tricks that make DQN practical.

Scientific thinking

The report encourages controlled comparisons rather than anecdotal claims from a single run.

Deliverables

Comparison plots for hyperparameters plus a written report with insights.

Starter Code and Training Notes

Template files, video recording, and practical advice

Starter files

The README points students to a starter template in lunar_lander.py and a helper file utils.py.

Video recording snippet

```
env = gym.make('LunarLander-v3', render_mode='rgb_array')
env = RecordVideo(env, 'videos/',
                  episode_trigger=lambda x: x % 50 == 0)
```

The note explains that recording every 50th episode keeps disk usage manageable during long runs.

Training advice

- Expect instability and use moving averages to reveal trends
- Save checkpoints every 50–100 episodes
- Use visualisation to debug what the agent is learning
- Be patient: good performance may take 300–500 episodes

Submission Requirements and Grading Summary

What students must turn in

Submission package (Don’t use ZIP file)

- All Python source files (.py)
- Trained model weights (.pt or .pth)
- Plots and figures (.png or .pdf)
- 3–5 videos or GIFs showing agent behaviour
- Written report (.pdf)

100-point rubric

Part A: Baseline	**15**
Part B: Implementation	**40**
Part C: Training & Analysis	**30**
Part D: Experimentation & Report	**15**
Total	**100**

Balanced emphasis: implementation + evidence + reflection